

Using character-LSTM for classifying ethnicity from last name

- In this homework, you are asked to use build a character-LSTM model to classify last name.
- Here is a sample LSTM model, you are free to choose your own model.
- **Challenge: what's the highest test accuracy can you get?**
- Of course you are not allowed to use test data for training

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn.utils.rnn import pad_sequence

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)

class CharacterLSTMModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim, activation = 'auto', pooled = False, dropout
        """
        Args:
        - vocab_size: Number of unique characters.
        - embedding_dim: Size of the embedding vectors.
        - hidden_dim: Number of hidden units in the LSTM.
        - output_dim: Size of the output label vector.
        """
        super(CharacterLSTMModel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=num_layers, bidirectional = False, dropout = dropout

        if activation == 'auto':
            self.activation = nn.LeakyReLU()
        else:
            self.activation = activation

        # 2* is needed for bidirectional
        #self.fc = nn.Linear(2*hidden_dim, output_dim)
        self.fc = nn.Sequential(nn.Linear(hidden_dim, 64))
        if self.activation is not None:
            self.fc.append(self.activation)
        self.fc.append(nn.Linear(64, 32))
        if self.activation is not None:
            self.fc.append(self.activation)
        self.fc.append(nn.Linear(32, output_dim))

        self.softmax = nn.Softmax(dim=1)
        self.pooled = pooled

    def forward(self, x):
        # x: (batch_size, seq_len)

        #print("x shape = ", x.shape)
        embedded = self.embedding(x) # (batch_size, seq_len, embedding_dim)
        #print("embedded shape = ", embedded.shape)

        lstm_out, (h,c) = self.lstm(embedded) # lstm_out: (batch_size, seq_len, hidden_dim)
        #print("lstm_out shape = ", lstm_out.shape)

        # Max pooling over the sequence
        if self.pooled:
            pooled, _ = torch.max(lstm_out, dim=1) # (batch_size, hidden_dim)
        else: # take the last output
            pooled = lstm_out[:, -1, :] # (batch_size, hidden_dim)
        #print("pooled shape = ", pooled.shape)
        #print("FC shape = ", self.fc.weight.shape)

        # Fully connected layer
        logits = self.fc(pooled) # (batch_size, output_dim)

        # Softmax for probabilities
        probabilities = self.softmax(logits) # (batch_size, output_dim)

        return probabilities
```

```
def create_vocab(strings):
    chars = set("".join(strings)) # Get all unique characters in the dataset
    vocab = {char: idx + 1 for idx, char in enumerate(sorted(chars))} # Map chars to indices (start from 1)
    vocab["<PAD>"] = 0 # Add padding character
    return vocab

def encode_strings(strings, vocab):
    sequences = [[vocab[char] for char in string] for string in strings]
    return sequences

def pad_sequences(sequences, pad_idx=0):
    tensor_seqs = [torch.tensor(seq) for seq in sequences]
    padded_seqs = pad_sequence(tensor_seqs, batch_first=True, padding_value=pad_idx)
    return padded_seqs

def strings2seq(strings, vocab):
    s = encode_strings(strings, vocab)
    return pad_sequences(s, pad_idx = vocab["<PAD>"]).to(device)
```

▼ prepare data

- census_lastname.csv can be downloaded from Canvas, or get the latest from [Census](#)
- Import name2race data, do train/valid/test split

```
import pandas as pd
import numpy as np

df = pd.read_csv("/content/Names_2010Census.csv")
df['name'] = df['name'].apply(lambda x: "nan" if x is np.nan else x.lower()) # one of the names is "nan"
races = df.columns[5:]
df[races] = df[races].apply(lambda col: pd.to_numeric(col, errors="coerce").fillna(0)) # get rid of races with "(S)" and
df[races] = df[races].apply(lambda x: x/df[races].values.sum(axis = 1))
data = df['name']
labels = torch.tensor(df[races].values, dtype = torch.float32)
vocab = create_vocab(data)
```

```
# train/valid/test split using random_state = 42
# add your code here to get df_train, df_valid, df_test
from sklearn.model_selection import train_test_split

df_train, df_test = train_test_split(df, test_size = 0.2, random_state = 42)
df_train, df_valid = train_test_split(df_train, test_size = 0.2, random_state = 42)
```

```
from torch.utils.data import DataLoader, TensorDataset, random_split

padded_sequences_train = strings2seq(df_train['name'].values, vocab)
labels_train = torch.tensor(df_train[races].values, dtype = torch.float32)

padded_sequences_valid = strings2seq(df_valid['name'].values, vocab)
labels_valid = torch.tensor(df_valid[races].values, dtype = torch.float32)

padded_sequences_test = strings2seq(df_test['name'].values, vocab)
labels_test = torch.tensor(df_test[races].values, dtype = torch.float32)

batch_size = 1024
torch.manual_seed(42)

train_dataset = TensorDataset(padded_sequences_train, labels_train)
test_dataset = TensorDataset(padded_sequences_test, labels_test)
valid_dataset = TensorDataset(padded_sequences_valid, labels_valid)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle = True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle = True)
```

▼ Optimize your model

- try different embedding dimension and hidden dimension and decide on the parameters that gives the best performance
- optional: try
 - different model (e.g., GRU),
 - multiple layers of LSTM,

- bidirectional model
- different loss function

```
import time
torch.manual_seed(0)
vocab_size = len(vocab)
print("vocab_size = ", vocab_size)

embedding_dim = 16
hidden_dim = 64
output_dim = labels.shape[1] # Number of classes = # of races

# Initialize the model
model = CharacterLSTMModel(vocab_size, embedding_dim, hidden_dim, output_dim, activation = 'auto', dropout = 0.5, pooled = False)

# try different loss function
criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.002)
epochs = 20

with torch.no_grad():
    loss_valid = torch.mean(torch.tensor([criterion(torch.log(model(inputs))).to(device),
                                          criterion(model(torch.tensor(targets.to(device))).to(device)) for inputs, targets in valid_loader])).to(device)

print(f'before training, Validation Loss: {loss_valid.item():.4f}')

def get_acc(dist1, dist2):
    return (torch.max(dist1, dim = 1).indices == torch.max(dist2, dim = 1).indices).sum()/dist1.shape[0]

start_time = time.time()
for i in range(epochs):
    for inputs, targets in train_loader:
        targets = targets.to(device)
        optimizer.zero_grad()
        out = model(inputs)
        if isinstance(criterion, nn.KLDivLoss) or isinstance(criterion, nn.CrossEntropyLoss): # both expect logit
            out = torch.log(out)
        if isinstance(criterion, nn.CrossEntropyLoss):
            targets = nn.Softmax(dim = 1)(100.*targets) # make into 0/1 for cross entropy loss
        loss = criterion(out, targets)
        print(f'Epoch [{i+1}/{epochs}], Batch Loss: {loss.item():.4f}', end = '\r')
        loss.backward()
        optimizer.step()

    with torch.no_grad(): # monitor validation error
        loss = []; acc = []
        for inputs, targets in valid_loader:
            targets = targets.to(device)
            p = model(inputs)
            if isinstance(criterion, nn.KLDivLoss):
                p = torch.log(p)
            loss.append(criterion(p, targets))
            acc.append(get_acc(p, targets))
        loss = torch.mean(torch.tensor(loss))
        acc = torch.mean(torch.tensor(acc))
        print(f'Epoch [{i+1}/{epochs}], Validation Loss: {loss.item():.4f}, acc:{acc.item():.4f}')

end_time = time.time()
print("time = ", end_time - start_time)
```

```
vocab_size = 28
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/rnn.py:990: UserWarning: dropout option adds dropout after all but last recurrent layer
super().__init__("LSTM", *args, **kwargs)
before training, Validation Loss: 1.8619
Epoch [1/20], Validation Loss: 1.3562, acc:0.8219
Epoch [2/20], Validation Loss: 1.3554, acc:0.8226
Epoch [3/20], Validation Loss: 1.3244, acc:0.8283
Epoch [4/20], Validation Loss: 1.3220, acc:0.8308
Epoch [5/20], Validation Loss: 1.3061, acc:0.8359
Epoch [6/20], Validation Loss: 1.3004, acc:0.8411
Epoch [7/20], Validation Loss: 1.2964, acc:0.8429
Epoch [8/20], Validation Loss: 1.2860, acc:0.8464
Epoch [9/20], Validation Loss: 1.2964, acc:0.8495
Epoch [10/20], Validation Loss: 1.2858, acc:0.8531
Epoch [11/20], Validation Loss: 1.2903, acc:0.8528
Epoch [12/20], Validation Loss: 1.2906, acc:0.8531
Epoch [13/20], Validation Loss: 1.2893, acc:0.8545
Epoch [14/20], Validation Loss: 1.2908, acc:0.8548
Epoch [15/20], Validation Loss: 1.2799, acc:0.8571
Epoch [16/20], Validation Loss: 1.2781, acc:0.8576
Epoch [17/20], Validation Loss: 1.2738, acc:0.8584
Epoch [18/20], Validation Loss: 1.2817, acc:0.8587
Epoch [19/20], Validation Loss: 1.2682, acc:0.8594
Epoch [20/20], Validation Loss: 1.2669, acc:0.8587
time = 350.0756902694702
```

How many internal parameters does your best model have?

```
for name, param in model.named_parameters():
    print(name, param.shape)
```

```
embedding.weight torch.Size([28, 16])
lstm.weight_ih_10 torch.Size([256, 16])
lstm.weight_hh_10 torch.Size([256, 64])
lstm.bias_ih_10 torch.Size([256])
lstm.bias_hh_10 torch.Size([256])
fc.0.weight torch.Size([64, 64])
fc.0.bias torch.Size([64])
fc.2.weight torch.Size([32, 64])
fc.2.bias torch.Size([32])
fc.4.weight torch.Size([6, 32])
fc.4.bias torch.Size([6])
```

Report test accuracy

```
model.eval()
running_loss = 0.0
running_acc = 0.0
total = 0

with torch.no_grad():
    for inputs, targets in test_loader:      # train_loader or test_loader
        inputs = inputs.to(device)
        targets = targets.to(device)

        outputs = model(inputs)

        # Handle KLDivLoss
        if isinstance(criterion, nn.KLDivLoss):
            outputs = torch.log(outputs)

        loss = criterion(outputs, targets)
        acc = get_acc(outputs, targets)

        # Accumulate, weighted by batch size
        batch_size = targets.size(0)
        running_loss += loss.item() * batch_size
        running_acc += acc * batch_size
        total += batch_size

loss = running_loss / total
acc = running_acc / total

print(f"Loss: {loss:.4f}, Acc: {acc:.4f}")
```

```
Loss: 1.2669, Acc: 0.8588
```

test on a few names

```
def score(names, vocab):
    padded_sequences = strings2seq(names, vocab)
    with torch.no_grad():
        return model(padded_sequences)

last_names_by_group = {
    "white": ["Smith", "Johnson", "Miller", "Anderson", "Brown"],
    "asian": ["Lee", "Chen", "Nguyen", "Kim", "Wang"],
    "hispanic": ["Gonzalez", "Rodriguez", "Martinez", "Hernandez", "Lopez"],
    "black": ["Washington", "Jefferson", "Brown", "Robinson", "Jackson"],
    "api": ["Tupou", "Fisiina", "Maka", "Taumoepeau", "Langi"],
    "arabic": ["Haddad", "Nasser", "Fahmy", "Khalil", "Abbasi"]
}

names = []; eth = []
for k in last_names_by_group.keys():
    name_list = last_names_by_group[k]
    names = names + name_list
    eth = eth + [k for i in range(len(name_list))]
names = [n.lower() for n in names]
scores = score(names, vocab).cpu().numpy()

columns = [x[3:] for x in races]
df_score = pd.DataFrame(scores, columns = races)
df_predict = pd.DataFrame({'name':names, "ethnicity":eth, "predicted":[columns[np.argmax(s)] for s in scores]})
```

```
pd.concat([df_predict, df_score], axis = 1)
```

	name	ethnicity	predicted	pctwhite	pctblack	pctapi	pctaian	pct2prace	pcthispanic
0	smith	white	white	0.946101	4.455224e-02	0.002522	0.006115	7.797559e-05	0.000632
1	johnson	white	white	0.767243	1.603687e-01	0.010129	0.058124	1.916483e-03	0.002219
2	milller	white	white	0.979116	1.757465e-02	0.000353	0.002469	6.674824e-06	0.000480
3	anderson	white	white	0.910208	5.829493e-02	0.005320	0.019538	3.437888e-04	0.006295
4	brown	white	white	0.946230	4.346135e-02	0.001047	0.008590	5.328652e-05	0.000618
5	lee	asian	white	0.869288	7.800020e-02	0.027446	0.019287	1.480054e-03	0.004499
6	chen	asian	api	0.441021	4.986278e-02	0.453988	0.005282	2.307489e-03	0.047538
7	nguyen	asian	api	0.201489	1.194072e-01	0.638712	0.013613	2.063297e-02	0.006145
8	kim	asian	white	0.889392	1.693518e-02	0.088516	0.003700	6.040660e-04	0.000853
9	wang	asian	api	0.127709	2.241457e-01	0.585620	0.014576	4.524176e-02	0.002707
10	gonzalez	hispanic	hispanic	0.000097	9.882108e-07	0.000764	0.000001	2.783734e-11	0.999137
11	rodriguez	hispanic	hispanic	0.002422	6.605555e-06	0.001411	0.000010	2.727508e-10	0.996151
12	martinez	hispanic	hispanic	0.001300	1.356364e-05	0.001632	0.000012	7.021962e-10	0.997043
13	hernandez	hispanic	hispanic	0.003047	2.076883e-05	0.002491	0.000035	5.041268e-09	0.994406
14	lopez	hispanic	hispanic	0.035241	3.107423e-03	0.075766	0.000939	4.676306e-06	0.884941
15	washington	black	white	0.897785	2.395760e-02	0.032376	0.030382	3.071009e-04	0.015192
16	jefferson	black	white	0.952623	1.656198e-02	0.005176	0.021139	1.886372e-04	0.004311
17	brown	black	white	0.946230	4.346135e-02	0.001047	0.008590	5.328652e-05	0.000618
18	robinson	black	white	0.978114	1.181022e-02	0.001196	0.007805	3.245409e-05	0.001043
19	jackson	black	white	0.889312	9.751949e-02	0.001360	0.011014	8.183553e-05	0.000713
20	tupou	api	white	0.892582	5.897569e-02	0.031558	0.012948	8.367220e-04	0.003100
21	fisiina	api	white	0.939157	4.818049e-03	0.018187	0.001357	1.559900e-05	0.036466
22	maka	api	white	0.939661	1.862929e-02	0.033179	0.003077	1.536628e-04	0.005300
23	taumoepeau	api	api	0.148647	2.979101e-02	0.652052	0.031445	3.077222e-04	0.137757
24	langi	api	white	0.941864	7.212122e-03	0.037452	0.001585	5.408478e-05	0.011833
25	haddad	arabic	white	0.883464	7.630756e-02	0.013851	0.023945	1.053349e-03	0.001379
26	nasser	arabic	white	0.990688	6.924729e-03	0.000149	0.002109	2.859891e-06	0.000128
27	fahmy	arabic	white	0.813488	1.605437e-01	0.003688	0.018671	3.722933e-04	0.003237
28	khalil	arabic	white	0.854682	3.069909e-02	0.098850	0.012685	2.131671e-03	0.000952
29	abbasi	arabic	white	0.951671	1.048384e-02	0.025727	0.003182	9.355760e-05	0.008842



Test your last name

```
columns[np.argmax(score(["ma"], vocab))]
```

'white'

开始借助 AI 编写或生成代码。

